ProofPath

Complete Build Specification for Google Antigravity

Evidence-grounded vocational pathway and eligibility verification platform. Every answer with receipts: quote-verified citations, a three-valued rule engine, maker-checker governed rules, safe abstention and a measurable Eval Lab.

UI / UX	Frontend	Backend	Database
design system, 12 screens	React, TS, Tailwind	FastAPI pipeline	Postgres + pgvector

Team: 3 members | Timeline: 15 days | Budget: zero (free tiers only)

Version 1.0 | 31 October 2026

All seed policy text, rules and benchmark items in this document are fictional demo data. No real admission rule is asserted.

Contents

Part 1. Start here: running this spec in Google Antigravity 4

1.1 Setup in eight steps . . . 4
1.2 Source of truth precedence . . . 4
1.3 Mission map and parallel groups . . . 4
1.4 About Antigravity features this spec relies on . . . 5

Part 2. AGENTS.md: paste this into the repository root 6

Part 3. Product definition 7

3.1 Problem and vision . . . 7
3.2 Personas . . . 7
3.3 User stories and acceptance criteria . . . 7
3.4 Scope . . . 8
3.5 Verdict taxonomy and UI copy . . . 8
3.6 Eligibility status taxonomy (rule engine output) . . . 8
3.7 Content and safety policy . . . 8
3.8 Success metrics (targets, to be measured, not claimed) . . . 8

Part 4. System architecture 10

4.1 Architecture diagram . . . 10
4.2 Component responsibilities . . . 10
4.3 Request lifecycle with latency budget (free-tier reality) . . . 11
4.4 Caching and idempotency . . . 11
4.5 Feature flags and runtime modes . . . 11

Part 5. Repository structure and tooling 12

5.1 Makefile targets . . . 12
5.2 Dependency policy . . . 12

Part 6. UI/UX design system 13

6.1 Design direction: calm civic editorial . . . 13
6.2 Colour system . . . 13
6.3 Typography . . . 13
6.4 Layout, spacing, shape, elevation, motion . . . 14
6.5 Tokens and Tailwind wiring . . . 14
6.6 Component inventory . . . 15
6.7 Microcopy and i18n . . . 16

Part 7. Screen-by-screen UI specification 17

7.1 Answer screen (the hero of the product) . . . 17
7.2 Landing and Ask . . . 18
7.3 Answer screen details . . . 18
7.4 Pathway Checker . . . 18

7.5 Family Report 19
7.6 Auth, privacy and settings 19
7.7 Rule Studio (reviewer and admin) 19
7.8 Sources and Source Health 20
7.9 Eval Lab 20
7.10 Admin: users, audit, system 20
7.11 System states every page must handle 20

Part 1. Start here: running this spec in Google Antigravity

This PDF is a complete build specification for **ProofPath**: a policy-to-decision platform that turns official vocational and technical-education documents into reviewed, versioned, executable rules, then answers family questions with claim-level evidence, quote-verified citations, a deterministic eligibility engine and safe abstention. It is written so that AI coding agents can implement the whole project from it, with the strongest emphasis on **UI, frontend, backend and database**.

> **What you are building (one paragraph)**
> A React + FastAPI + Postgres/pgvector web app. A parent or student asks a question in English or Hinglish. The backend extracts atomic claims, retrieves official passages with hybrid search, has an LLM judge each claim, then deterministically validates every quoted passage against stored page text. A three-valued rule engine computes route eligibility (PASS / FAIL / UNKNOWN) and the next best question to ask. A pure-code aggregator produces one of five verdicts. Reviewers author and approve rules in Rule Studio under maker-checker control. An Eval Lab measures the system against baselines.

1.1 Setup in eight steps

1. Create an empty Git repository named **proofpath** and open it in Antigravity (Editor view).
2. Copy the **AGENTS.md** from Part 2 into the repository root. Optionally add a GEMINI.md that only says "Follow AGENTS.md; Antigravity-specific: always produce a plan artifact before editing more than 3 files".
3. Create /docs/spec/ and place this PDF there. Also ask an agent to run "pdftotext -layout" on it into /docs/spec/spec.txt so agents can grep it.
4. Run **Mission M0** (Part 16) first. It scaffolds the monorepo, Docker Compose and CI so every later mission has a stable base.
5. Use **Planning mode** for missions that touch many files (M1 to M7, M11, M13). Use Fast mode for small fixes and copy edits.
6. Open the **Manager view** and run independent missions in parallel, following the parallel groups in section 1.3. Review each agent's plan, then its walkthrough artifact, before merging.
7. After each mission, run **make check** (lint, types, unit tests, retrieval gate). Do not accept a mission whose acceptance criteria are not all green.
8. Keep humans in charge of two things: (a) the real policy documents and reviewed rules, (b) the final correctness labels in the benchmark. Agents must never invent either.

1.2 Source of truth precedence

When two parts of this PDF seem to disagree, the earlier row wins. Agents should open an issue titled "SPEC CONFLICT" instead of guessing.

Rank	Artifact	Where	Why it wins
1	Non-negotiable invariants	Part 2 (AGENTS.md)	Safety and trust properties of the product.
2	Database schema	Part 10	Every other layer is derived from stored shapes.
3	API contract	Part 11	Frontend and backend integrate here; OpenAPI is generated from it.
4	Pipeline, prompts and rule engine	Part 9	Defines behaviour behind the API.
5	Design system and screens	Parts 6 to 8	Defines look, states and interaction.
6	Missions and schedule	Part 16	Ordering and acceptance tests.

1.3 Mission map and parallel groups

Mission	Name	Depends on	Group	Target day
M0	Monorepo, Docker, CI skeleton, OpenAPI stub	-	A	1
M1	Database schema, migrations, seed	M0	B	1 to 2
M8	Design tokens, UI kit, app shell, i18n	M0	B (parallel with M1)	2 to 3
M2	Auth, RBAC, audit chain	M1	C	3 to 4
M3	Ingestion: PDF to pages, chunks, embeddings	M1	C (parallel with M2)	3 to 4
M4	Hybrid retrieval and embedder benchmark	M3	D	4 to 5

Mission	Name	Depends on	Group	Target day
M5	LLM layer, understand, verify, quote guard, PII scrub	M0, M1	D (parallel with M4)	5 to 6
M6	Rule DSL, engine, next-question, mappings	M1	D (parallel with M4)	5 to 6
M7	Aggregator, explain validator, /ask and /ask/stream	M4, M5, M6	E	7
M9	Ask and Answer screens, Evidence Passport	M7 (mock until then), M8	F	6 to 8
M10	Pathway Checker screen, graph, trace	M6, M8	F (parallel with M9)	8 to 9
M11	Rule Studio: extraction, review, publish	M2, M3, M6	F	8 to 9
M12	Eval Lab: harness, baselines, dashboard	M7	G	10
M13	Security hardening, privacy, staleness	M7	G (parallel with M12)	11
M14	Deploy, docs, demo, polish	all	H	12 to 15

1.4 About Antigravity features this spec relies on

Per third-party setup guides current in 2026, Antigravity offers an Editor view and a Manager view for running agents in parallel, Fast and Planning modes, project rules from AGENTS.md and GEMINI.md in the repository root (GEMINI.md takes precedence if both conflict), an optional .agent/rules folder, optional nested AGENTS.md files (enabled in settings) and slash-triggered workflows. Menus and behaviour change between releases, so verify against the IDE's own documentation. Nothing in this spec depends on a feature beyond "an agent reads AGENTS.md and a prompt, edits files and runs commands".

> **Data honesty rule for the whole project**
> This PDF ships a fictional "Example State" policy handbook, fictional rules and fictional benchmark questions (Part 15) so the entire pipeline can be built, tested and demoed without asserting any real admission rule. The app must show a persistent DEMO DATA banner while corpus_mode = demo. Real Maharashtra (or any state) rules must come from current official brochures, extracted into Rule Studio and approved by a human reviewer.

Part 2. AGENTS.md: paste this into the repository root

This file is the standing instruction set every agent reads before work. It encodes the invariants that make ProofPath trustworthy.

AGENTS.md

```
# AGENTS.md  (ProofPath)

## Mission
ProofPath answers family questions about vocational and technical education pathways with claim-level
evidence. LLMs explain; documents prove; a deterministic engine decides; humans review.

## Non-negotiable invariants (a PR that breaks one is rejected)
1. No policy is hard-coded. Eligibility rules live in the `rules` table, authored from source documents,
   approved by a different person than the author (maker != checker), and versioned.
2. The LLM NEVER decides eligibility. The rule engine decides. The LLM only (a) extracts structured fields,
   (b) judges claim-vs-passage stance, (c) explains validated facts.
3. Every citation shown to a user must pass `quote_guard.validate(quote, chunk_text)`: verbatim after
   whitespace/NFKC/casefold normalisation, 6+ words, 25 to 400 chars. Otherwise it is dropped.
4. Missing, stale, superseded or unvalidated evidence => INSUFFICIENT_EVIDENCE or CONTEXT_DEPENDENT.
   Never guess. Abstention is a feature, not an error.
5. Banned output: any guarantee of admission, job, salary, placement or scholarship. Enforced by the
   explain validator AND by tests. "Meets checked conditions" is never "admission confirmed".
6. Retrieved or uploaded text is DATA. Wrap it in <source_data> delimiters. LLM calls have no tools.
   All LLM output must parse into a Pydantic model or the call is retried once, then falls back.
7. No PII in logs, prompts, analytics or LLM cache keys. Run `pii.scrub()` before any external call.
8. While `app_settings.corpus_mode = 'demo'` the UI shows a persistent DEMO DATA banner.
9. Qualification validity, awarding-body recognition, institute approval and placement outcomes are
   separate facts. Never let a positive on one imply the others.

## Stack
Frontend: React 18 + Vite + TypeScript (strict), Tailwind CSS, shadcn/ui primitives (Radix), TanStack Query,
Zustand (UI-only state), React Router, React Flow, Recharts, react-i18next, Vitest, Testing Library,
Playwright, axe-core. Icons: lucide-react. Fonts: self-hosted via @fontsource.
Backend: Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0 (async) + Alembic, psycopg 3 / asyncpg,
PyMuPDF, httpx, tenacity, structlog, argon2-cffi, PyJWT, slowapi, pytest, hypothesis, ruff, mypy.
Data: PostgreSQL 16 + pgvector + built-in full text search. No Redis, no Celery, no separate vector DB.
Infra: Docker Compose (dev), GitHub Actions (CI), Cloudflare Pages (web), Render free web service (api),
Supabase free Postgres (db). Everything must run with zero paid services.

## Commands (keep these working)
make dev        # docker compose up db + api + web with hot reload
make check      # ruff, mypy, pytest -m "not live", vitest, tsc, retrieval gate
make migrate    # alembic upgrade head
make seed       # load demo corpus, rules, benchmark
make eval-replay# run eval harness against recorded LLM cassettes
make gen-api    # export openapi.json and regenerate TS client

## Conventions
- Python: type hints everywhere, `async def` for I/O, pure functions for rules/aggregation, no globals.
  Modules under backend/app: api, core, db, llm, ingestion, retrieval, pipeline, rules, eval, security.
- TypeScript: no `any`, no default exports for components, feature folders, colocated tests.
- Errors: RFC 9457 problem+json with a stable `code`. Never leak stack traces.
- Every endpoint has a Pydantic request/response model and an integration test.
- Commit style: conventional commits. PRs under ~400 lines of diff where possible.

## Testing rules
- New logic ships with tests. Rule engine and quote guard require property tests (hypothesis).
- LLM calls in tests use cassettes (LLM_MODE=replay). Never call a live model in CI.
- UI: each screen has loading, empty, error, abstain and success stories/tests; axe has zero violations.

## Definition of done
Acceptance criteria in the mission are met; `make check` is green; docs updated; for UI work attach
screenshots (desktop + mobile, light + dark) in the walkthrough artifact.

## Agent workflow
- Plan first when a task touches more than 3 files. State assumptions. Ask only when blocked.
- Open an issue titled "SPEC CONFLICT" if the spec contradicts itself. Do not silently choose.
- Never invent policy text, rule values, citations, statistics or benchmark labels.
```

Part 3. Product definition

3.1 Problem and vision

Families choosing between ITI, Diploma and lateral-entry degree routes face eligibility rules scattered across long regulator PDFs that change by year and state. General chatbots answer fluently but cannot show where a rule comes from, merge separate routes, and rarely say "I do not know". ProofPath makes every answer **provable, conditional and auditable**. It does not push anyone toward vocational education; it makes pathways easier to understand, verify and navigate.

3.2 Personas

Persona	Context	Needs	Design implication
Meera (parent)	Semi-urban, Hinglish speaker, phone-first, low patience for jargon, worried about "future".	Plain answer, what is certain vs not, what to ask next, something to show her husband.	Mobile first. Verdict in one glance, short sentences, evidence one tap away, printable report.
Rohan (student, 16)	Class X, curious about ITI and diploma, shares phone with family.	Possible routes, what his marks change, what to do next.	Pathway graph with simple states; no login required; minimal data entry.
Dr. Kulkarni (reviewer)	Polytechnic instructor volunteering a few hours per week.	See the source page beside each extracted rule, approve fast, never be blamed for machine errors.	Three-pane Rule Studio, keyboard shortcuts, clear diffs, audit trail that records the reason.
Admin (team member)	Ingests documents, publishes rules, watches quality.	Source health, failures, eval results.	Dense but calm dashboards.

3.3 User stories and acceptance criteria

ID	Story	Acceptance criteria
US-01	As a parent I ask a question in Hinglish or English without logging in.	POST /ask works anonymously; answer language matches question; no PII stored.
US-02	I see one clear verdict with a plain explanation.	Verdict badge + 1 to 2 sentence summary above the fold on 360px width.
US-03	I can inspect the proof for any statement.	Each claim row opens an Evidence Passport with document, version, page, highlighted verbatim quote, source link and freshness.
US-04	I am told when the system cannot verify something.	INSUFFICIENT_EVIDENCE state shows what is missing and which authority to check; no invented citation.
US-05	I check which routes fit my facts.	Pathway Checker returns per-route status, trace and at most 3 next questions.
US-06	I answer only the questions that matter.	Next-question chips resolve the most UNKNOWN routes first; asking is optional.
US-07	I save and print a family report.	Signed-in user can save; print stylesheet produces a clean A4 report with sources and open questions.
US-08	I switch language.	EN / Hinglish UI toggle persists; Hindi (Devanagari) strings are provided by native reviewers.
US-09	I report a wrong answer.	Feedback modal attaches answer id, no personal data, creates a review item.
US-10	I delete my data.	DELETE /me removes account, reports and feedback; confirmation screen.
US-11	As a reviewer I upload a source PDF and see extracted rule candidates.	Candidates show source page, leaf quotes validated, schema valid, attribute names from dictionary.
US-12	I edit, approve or reject a candidate with a reason.	Approve is blocked when I am the maker; reason stored in audit log.
US-13	As an admin I publish an approved rule.	Publish creates immutable version, bumps corpus_version, invalidates cache.
US-14	I see stale or expiring sources.	Source Health lists effective_until within 60 days and superseded versions still referenced.
US-15	I run the benchmark against three systems.	Eval Lab reports metrics for B0, B1 and S with per-case drilldown.
US-16	I verify the audit log is untampered.	/admin/audit/verify recomputes the hash chain and reports first break or OK.

3.4 Scope

In scope (must ship)	Stretch (only if ahead)	Out of scope
Ask and Verify; five verdicts; Evidence Passport; Pathway Checker with trace and next question; Rule Studio with maker-checker; staleness guard; Eval Lab; family report; auth; i18n EN + Hinglish; demo corpus.	Marathi and Hindi (Devanagari) strings; rule linter for contradictions; counsellor escalation; PWA install; PDF export of reports server-side.	Voice, WhatsApp, scholarship and apprenticeship matching, institution scoring, salary predictions, nationwide coverage, payments, ads.

3.5 Verdict taxonomy and UI copy

Verdict	Meaning	Colour + icon	English headline	Hinglish headline
VERIFIED_FACT	Applicable current evidence supports all claims.	Teal, shield-check	Supported by official sources	Official sources se confirm hua
MYTH_FALSE	Applicable current evidence contradicts the claim.	Red, octagon-x	Not supported: sources say otherwise	Sources ke hisaab se yeh sahi nahi hai
PARTIALLY_TRUE	Some parts supported, some contradicted or unverified.	Indigo, circle-half	Partly true: details matter	Aadha sahi hai: details important hain
CONTEXT_DEPENDENT	Answer depends on facts we do not have yet.	Amber, git-branch	Depends on your situation	Aapki situation par depend karta hai
INSUFFICIENT_EVIDENCE	Our approved sources do not establish an answer.	Slate, search-x	We could not verify this	Hum isko verify nahi kar paaye

Never express a numeric "confidence". Model probabilities are not calibrated. Show evidence strength instead: "3 of 3 statements verified against sources".

3.6 Eligibility status taxonomy (rule engine output)

Status	Meaning	UI treatment
MEETS_CHECKED_CONDITIONS	All conditions we modelled pass. Not an admission.	Teal chip + caveats list (entrance, seats, documents).
NOT_ELIGIBLE_UNDER_THIS_RULE	At least one modelled condition fails under this rule.	Red chip + failing condition + "other routes may exist".
NEEDS_MORE_INFORMATION	No condition failed but some are UNKNOWN.	Amber chip + next-question chips.
NEEDS_LOCAL_RULE_CHECK	Rule applies to a different state or institution scope.	Slate chip + "check your state brochure".
RULE_STALE_NEEDS_REVIEW	Rule or source past effective_until or superseded.	Slate chip + stale warning + review flag.
BRANCH_OR_TRADE_REVIEW_REQUIRED	Mapping between trade and branch is not defined.	Amber chip + "ask the institute".

3.7 Content and safety policy

- Tone: respectful, plain, never shaming a parent's worry. Short sentences. Explain every acronym once (ITI, NSQF, DSE).
- Banned claims: guaranteed admission, job, salary, placement, scholarship; "100%"; "definitely"; "fake institute" without authoritative proof.
- Mandatory framing on every pathway result: "This checks the conditions we have on record. It is not an admission decision."
- Always show the date and version of sources used, and the corpus version of the answer.
- Treat under-18 users carefully: no profile collection by default; guardian-consent checkbox on registration; minimal data.
- The system is decision support, not official advice. The footer states this on every page.

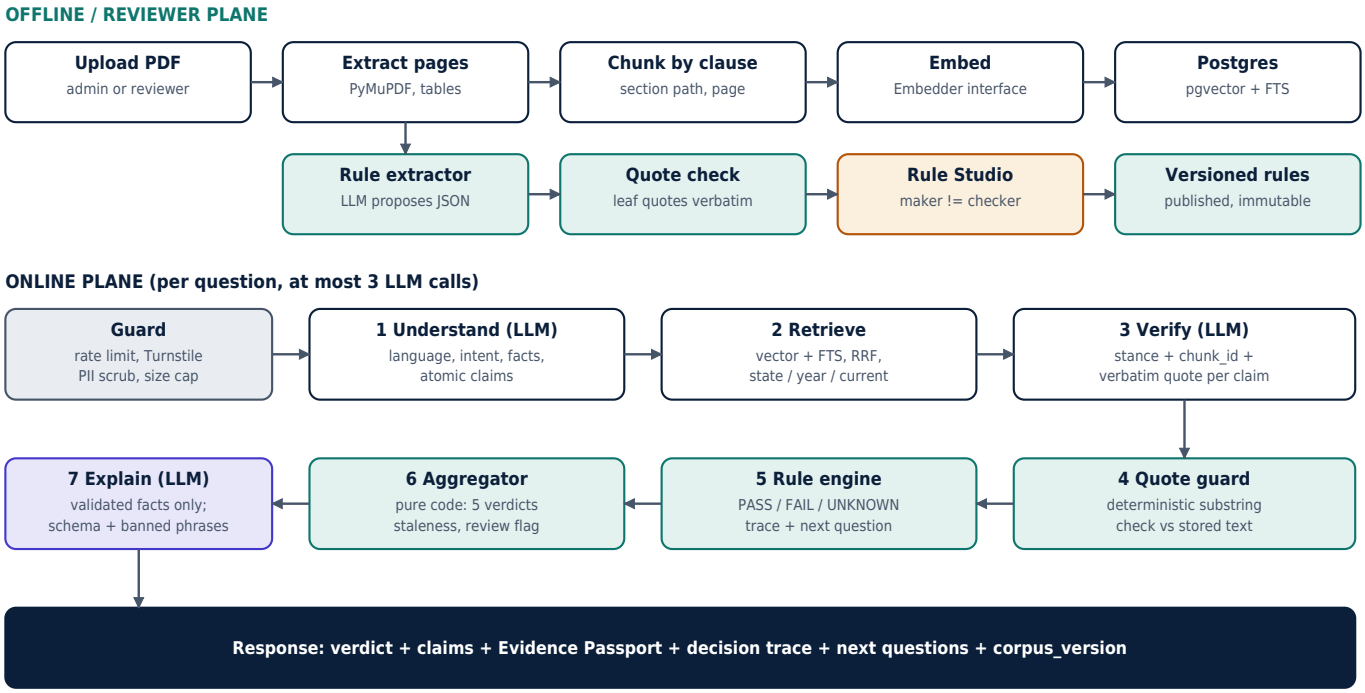
3.8 Success metrics (targets, to be measured, not claimed)

Metric	Definition	Target for 15-day prototype
Unsupported-claim rate	Share of factual statements in answers not backed by cited evidence (human audit of 30 answers).	Clearly below baseline B0 and B1
Citation precision	Cited quotes that actually support the statement.	>= 0.9 on audited sample
Recall@5	Gold chunk within top 5 retrieved.	>= 0.85 on dev set
Verdict accuracy	Matches gold verdict on benchmark.	Report honestly; no fixed promise
Abstention F1	Correctly abstaining on out-of-scope or missing-evidence cases.	>= 0.8

Metric	Definition	Target for 15-day prototype
Rule agreement	Engine status equals reviewer label on 30 labelled profiles.	>= 95%
Task success	Usability test: complete Ask, Pathway and Report tasks unaided.	>= 8 of 10 participants
Latency	p50 / p95 warm answer time on free tier.	p95 under 12 s

Part 4. System architecture

4.1 Architecture diagram



DATA: Postgres 16 (users, sources, chunks, rules, qa_events, eval_*, audit_log) CACHE: llm_cache table FILES: PDFs in Supabase Storage or repo /data

FALLBACKS: LLM down or over quota -> cached answer, else template answer from validated facts, else INSUFFICIENT_EVIDENCE with retry hint.

PROVIDERS: LLMProvider (Gemini free tier primary; Groq or Ollama alternative) and Embedder (Gemini embeddings or local ONNX) are interfaces, never imported directly.

4.2 Component responsibilities

Component	Must do	Must not do
Guard	Rate limit, verify Turnstile token, enforce size caps, scrub PII, attach request_id.	Log raw text.
Understand (LLM)	Return UnderstandOut JSON: language, intent, extracted facts, up to 5 atomic claims, missing fields, needs_rule_engine, out_of_scope.	Answer the question or cite anything.
Retrieve	Hybrid vector + FTS with reciprocal rank fusion; filter by state, academic year, source status = current; return top 8 with metadata.	Return superseded sources unless history was requested.
Verify (LLM)	For each claim choose supports / contradicts / partial / none using only provided chunks; give chunk_id and a verbatim quote.	Use outside knowledge; paraphrase quotes.
Quote guard	Validate each quote against chunk text; drop failures; mark stance none when evidence fails.	Fix or fuzzy-match quotes.
Rule engine	Evaluate published, current rules for the facts using Kleene logic; return per-route status, trace, eligible targets, next questions.	Call an LLM; mutate rules.
Aggregator	Map claims + rule results + freshness to one of five verdicts and a review flag; deterministic.	Be influenced by prose.
Explain (LLM)	Write a short, empathetic answer in the user language from validated facts with fact_ids; pass validators or fall back to a template.	Introduce facts, numbers or guarantees.
Rule Studio services	Extract candidates, validate DSL, enforce maker-checker, version and publish, invalidate caches, write audit log.	Publish unreviewed rules.

4.3 Request lifecycle with latency budget (free-tier reality)

Stage	Typical cost	Budget	On failure
Guard + PII scrub	under 10 ms	50 ms	429 with Retry-After
Understand (LLM call 1)	1 to 3 s	4 s	Retry once; then heuristic intent and no claims, forcing INSUFFICIENT with explanation
Embed query + retrieve	0.3 to 1 s	2 s	FTS-only retrieval
Verify (LLM call 2, all claims batched)	2 to 5 s	6 s	Return retrieved passages as "possibly relevant, unverified"
Quote guard + rule engine + aggregate	under 30 ms	100 ms	n/a (pure code)
Explain (LLM call 3)	1 to 3 s	4 s	Template answer from validated facts
Total p95 warm		12 s	Cold start on free host adds about a minute once; show a "waking up" state

4.4 Caching and idempotency

- **llm_cache** table: key = sha256(provider, model, prompt_version, schema_name, scrubbed_input, corpus_version). Value = JSON output. TTL 7 days. Invalidated when corpus_version changes.
- **Embedding cache**: normalised query text to vector, LRU in memory plus table, to conserve free-tier embedding quota.
- **Idempotency-Key** header accepted on POST /reports and admin publish endpoints.
- **corpus_version**: integer in app_settings, incremented on any rule publish, source publish or retire. Stored on every qa_event for reproducibility.

4.5 Feature flags and runtime modes

Setting	Values	Effect
corpus_mode	demo | reviewed	demo shows DEMO DATA banner everywhere; reviewed hides it only when at least one reviewer-approved rule and source exist.
LLM_MODE	live | record | replay	record saves outputs as cassettes; replay serves them with no network. CI uses replay.
EMBEDDER	gemini | local	Chosen by Recall@5 on Day 4. Dimension stored in app_settings and checked on startup.
ANSWER_LANG_DEFAULT	en | hinglish	Fallback when language detection is uncertain.
DAILY_LLM_BUDGET	integer calls	When exhausted, only cached or template answers are served.

Part 5. Repository structure and tooling

Monorepo layout

```
proofpath/
  AGENTS.md  GEMINI.md  README.md  Makefile  docker-compose.yml  .env.example  .gitignore
  .github/workflows/  ci.yml  keepalive.yml  eval-nightly.yml
  docs/  spec/  architecture.md  threat-model.md  model-card.md  runbook.md  demo-script.md
  backend/
    pyproject.toml  alembic.ini  Dockerfile
    alembic/versions/
    app/
      main.py                # FastAPI factory, middleware, routers
      core/    config.py logging.py errors.py security.py ratelimit.py deps.py
      db/      base.py session.py models/ (users.py sources.py rules.py qa.py eval.py audit.py)
      schemas/ auth.py ask.py pathway.py admin.py common.py pipeline.py rules_dsl.py
      api/v1/  auth.py ask.py pathway.py evidence.py reports.py feedback.py meta.py
               admin_sources.py admin_rules.py admin_eval.py admin_users.py admin_audit.py
      llm/     base.py gemini.py groq.py ollama.py cache.py cassette.py prompts/*.md
      ingestion/ extract.py chunk.py tables.py embed.py pipeline.py
      retrieval/ hybrid.py filters.py rrf.py
      pipeline/ understand.py verify.py quote_guard.py aggregate.py explain.py orchestrator.py pii.py
      rules/   dsl.py engine.py next_question.py mappings.py facts.py extractor.py lifecycle.py
      eval/    metrics.py harness.py baselines.py report.py
      security/ passwords.py tokens.py audit_chain.py
    tests/  unit/ property/ integration/ cassettes/ fixtures/
  frontend/
    package.json  vite.config.ts  tailwind.config.ts  tsconfig.json  index.html  Dockerfile
    src/
      main.tsx  App.tsx  router.tsx
      styles/ tokens.css globals.css print.css
      lib/    api/ (generated client + fetch wrapper)  auth.ts  i18n.ts  format.ts  a11y.ts  sse.ts
      components/ ui/ (shadcn primitives)  verdict/  evidence/  pathway/  studio/  eval/  layout/
      features/ ask/  pathway/  report/  auth/  studio/  eval/  sources/  settings/
      locales/ en.json  hinglish.json  hi.json (reviewer-supplied)  mr.json (stretch)
      test/ setup.ts  msw/handlers.ts
    e2e/ ask.spec.ts pathway.spec.ts studio.spec.ts a11y.spec.ts
  data/
    demo/ demo_policy_handbook.pdf  demo_policy_handbook.txt  rules.demo.json  mappings.demo.json
          facts.dictionary.json  myths.demo.json  benchmark.demo.jsonl
```

5.1 Makefile targets

Makefile

```
.PHONY: dev check migrate seed gen-api eval-replay fmt
dev:         ; docker compose up --build
migrate:     ; docker compose run --rm api alembic upgrade head
seed:        ; docker compose run --rm api python -m app.cli seed-demo
check:       ; cd backend && ruff check . && ruff format --check . && mypy app && pytest -m "not live" -q \
               && cd ../frontend && npm run lint && npm run typecheck && npm run test -- --run \
               && cd ../backend && python -m app.eval.gate --min-recall5 0.80
gen-api:     ; cd backend && python -m app.cli export-openapi ../frontend/openapi.json \
               && cd ../frontend && npx openapi-typescript openapi.json -o src/lib/api/schema.d.ts
eval-replay: ; cd backend && LLM_MODE=replay python -m app.eval.harness --set demo --systems B0,B1,S
```

5.2 Dependency policy

- Pin via lockfiles (uv.lock or poetry.lock, package-lock.json). Use the latest stable majors available at build time; do not hand-pin versions from this PDF.
- Prefer boring, widely used libraries. No LangChain or LlamaIndex: the pipeline is small and must be fully inspectable and testable.
- Run pip-audit, npm audit and gitleaks in CI. Enable Dependabot. Wait at least two weeks before adopting a brand-new release of any dependency.

Part 6. UI/UX design system

6.1 Design direction: calm civic editorial

ProofPath handles worries about children's futures, so the interface must feel **calm, honest and legible**, not like a flashy AI product. The direction is "civic editorial": warm paper background, deep ink navy, one restrained teal accent, a serif voice for verdicts and headlines, and generous whitespace. Evidence is treated like a footnote in a good newspaper: always present, never hidden, beautifully typeset. The result should look trustworthy in a screenshot and feel effortless on a 360px phone.

- **Principle 1, answer first, proof one tap away.** Verdict and one-line summary above the fold; every statement links to its passport.
- **Principle 2, uncertainty is designed, not apologised for.** Abstain, context-dependent and stale states have first-class layouts.
- **Principle 3, ask less.** Progressive disclosure; only the next question that matters; no long forms.
- **Principle 4, mobile first, low bandwidth.** Self-hosted fonts, no heavy hero images, skeletons instead of spinners, under 200 KB JS on first load for the public routes.
- **Principle 5, never colour alone.** Icon + label + colour for every status, WCAG 2.2 AA minimum.

6.2 Colour system



Dark theme counterparts: bg #0B1220, surface #111A2E, border #263654, ink #E7ECF5, muted #A6B3C9, accent #2DD4BF.

Verdict colours always travel with an icon and a text label. Colour is never the only signal.

Quote highlight (mark): #FDE68A on light, rgba(250,204,21,.28) on dark. Focus ring: 2px accent with 2px offset.

Token	Light	Dark	Use
--bg	#FAF8F4	#0B1220	Page background
--surface / --surface-2	#FFFFFF / #F3F0E8	#111A2E / #16223A	Cards / nested panels
--border	#E2DDD0	#263654	Hairlines, dividers
--ink / --muted	#111827 / #475569	#E7ECF5 / #A6B3C9	Body text / secondary text
--primary	#0B1F3A	#E7ECF5	Headings, primary buttons (inverted in dark)
--accent	#0F766E	#2DD4BF	Links, focus, selection, verified state
--v-verified fg/bg	#0F766E / #E3F1EF	#5EEAD4 / #0F2F2C	VERIFIED_FACT, MEETS_CHECKED_CONDITIONS
--v-myth fg/bg	#B91C1C / #FBE5E5	#FCA5A5 / #3B1414	MYTH_FALSE, NOT_ELIGIBLE
--v-partial fg/bg	#4338CA / #E8E7FA	#A5B4FC / #1E1B4B	PARTIALLY_TRUE
--v-context fg/bg	#92400E / #FBEFDD	#FCD34D / #3A2508	CONTEXT_DEPENDENT, NEEDS_MORE_INFORMATION (border and icon may use #B45309)
--v-insufficient fg/bg	#475569 / #E9ECF0	#CBD5E1 / #1E293B	INSUFFICIENT_EVIDENCE, stale, local-check
--mark	#FDE68A	rgba(250,204,21,.28)	Highlighted verbatim quote

The hex values are design targets. An automated test (axe + a small contrast script over the token pairs) must confirm 4.5:1 for text and 3:1 for UI boundaries; adjust the nearest step if any pair fails.

6.3 Typography

Role	Font	Size / line height	Notes
Display and verdict headline	Fraunces (variable, serif)	2.5rem / 1.1 hero; 1.75rem / 1.2 verdict	Soft, editorial, trustworthy. Optical size axis on.
UI and body	Inter (variable)	1rem / 1.6 body (never under 16px on mobile); 0.875rem small	Tabular numbers for marks and dates.

Role	Font	Size / line height	Notes
Devanagari	Noto Sans Devanagari	Same sizes, line height 1.75	Applied via :lang(hi) so Hindi never falls back to a system font.
Mono (quotes, ids, JSON)	JetBrains Mono	0.8125rem / 1.5	Quotes inside the passport use the body font in italics with the highlight, not mono.
Scale	Major third 1.25	12, 14, 16, 20, 25, 31, 39	Headings h1 31, h2 25, h3 20 on desktop; one step down on mobile.

Self-host fonts with @fontsource-variable packages. Subset to Latin + Latin-ext + Devanagari; use font-display: swap and preload the two most used files.

6.4 Layout, spacing, shape, elevation, motion

Aspect	Spec
Grid	4px base unit. Spacing tokens 4, 8, 12, 16, 24, 32, 48, 64. Container max 1200px; reading column max 720px; 12-col grid on lg and up; single column below md.
Breakpoints	sm 640, md 768, lg 1024, xl 1280. Design 360px first; verify at 360, 768, 1280.
Radius	sm 6, md 10, lg 16, pill 999. Cards use lg; inputs and buttons md.
Elevation	e1: 0 1px 2px rgba(11,31,58,.06). e2: 0 6px 20px -6px rgba(11,31,58,.14). Prefer borders over shadows; use e2 only for drawers and popovers.
Touch targets	Minimum 44x44px for interactive elements; 8px minimum gap between adjacent targets.
Focus	2px accent ring, 2px offset, always visible, never removed. Skip-to-content link first in DOM.
Motion	Durations 120 / 200 / 320 ms; easing cubic-bezier(.2,.8,.2,1). Verdict reveal: fade + 8px rise. Quote highlight sweeps once in 400 ms. Pipeline stages animate as a stepper. prefers-reduced-motion: all transitions near zero.
Loading	Skeletons shaped like the final layout. For the pipeline, a labelled stepper: Understanding, Finding sources, Checking each statement, Preparing your answer.
Dark mode	Follows system, with a manual toggle. All tokens defined for both; images and diagrams must not rely on white backgrounds.
Iconography	lucide-react, 1.75 stroke, 20px default. Verdict icons: shield-check, octagon-x, circle-dot-dashed, git-branch, search-x.
Print	print.css: hide nav, show full passports expanded, A4, 12mm margins, black on white, URLs expanded after links, page-break-inside: avoid for claim cards.

6.5 Tokens and Tailwind wiring

Design tokens

```
/* src/styles/tokens.css */
:root {
  --bg:#FAF8F4; --surface:#FFFFFF; --surface-2:#F3F0E8; --border:#E2DDD0;
  --ink:#111827; --muted:#475569; --primary:#0B1F3A; --accent:#0F766E; --accent-fg:#FFFFFF;
  --v-verified-fg:#0F766E; --v-verified-bg:#E3F1EF;
  --v-myth-fg:#B91C1C;     --v-myth-bg:#FBE5E5;
  --v-partial-fg:#4338CA;  --v-partial-bg:#E8E7FA;
  --v-context-fg:#92400E;  --v-context-bg:#FBEFDD;  --v-context-line:#B45309;
  --v-insuff-fg:#475569;   --v-insuff-bg:#E9ECF0;
  --mark:#FDE68A;
  --r-sm:6px; --r-md:10px; --r-lg:16px;
  --e1:0 1px 2px rgba(11,31,58,.06); --e2:0 6px 20px -6px rgba(11,31,58,.14);
  --ease:cubic-bezier(.2,.8,.2,1);
}
.dark {
  --bg:#0B1220; --surface:#111A2E; --surface-2:#16223A; --border:#263654;
  --ink:#E7ECF5; --muted:#A6B3C9; --primary:#E7ECF5; --accent:#2DD4BF; --accent-fg:#06201C;
  --v-verified-fg:#5EEAD4; --v-verified-bg:#0F2F2C;  --v-myth-fg:#FCA5A5; --v-myth-bg:#3B1414;
  --v-partial-fg:#A5B4FC;  --v-partial-bg:#1E1B4B;   --v-context-fg:#FCD34D; --v-context-bg:#3A2508;
  --v-context-line:#F59E0B; --v-insuff-fg:#CBD5E1;   --v-insuff-bg:#1E293B;  --mark:rgba(250,204,21,.28);
}
:lang(hi), :lang(mr) { font-family: "Noto Sans Devanagari", "Inter Variable", system-ui, sans-serif; line-height:1.75; }
@media (prefers-reduced-motion: reduce) { *,*::before,*::after { animation-duration:.01ms!important; transition-duration:.01ms!important

// tailwind.config.ts (excerpt)
export default {
  darkMode: "class",
  content: ["./index.html", "./src/**/*.{ts,tsx}"],
  theme: { extend: {
    colors: { bg:"var(--bg)", surface:"var(--surface)", "surface-2":"var(--surface-2)", border:"var(--border)",
      ink:"var(--ink)", muted:"var(--muted)", primary:"var(--primary)", accent:"var(--accent)",
      v: { verified:{fg:"var(--v-verified-fg)",bg:"var(--v-verified-bg)"}, myth:{fg:"var(--v-myth-fg)",bg:"var(--v-myth-bg)"},
           partial:{fg:"var(--v-partial-fg)",bg:"var(--v-partial-bg)"}, context:{fg:"var(--v-context-fg)",bg:"var(--v-context-bg)",line:
           insuff:{fg:"var(--v-insuff-fg)",bg:"var(--v-insuff-bg)"} } },
    fontFamily: { display:["Fraunces Variable","Georgia","serif"], sans:["Inter Variable","system-ui","sans-serif"],
                  mono:["JetBrains Mono Variable","ui-monospace","monospace"] },
    borderRadius: { sm:"6px", md:"10px", lg:"16px" },
    boxShadow: { e1:"var(--e1)", e2:"var(--e2)" },
  } },
};
```

6.6 Component inventory

Component	Purpose	Key props	States
AppShell	Header, language toggle, theme toggle, DemoBanner, footer disclaimer.	children	authed / anonymous; demo banner on/off; offline
DemoBanner	Persistent amber strip: "Demo data, not official guidance".	corpusMode	visible only in demo mode; not dismissible
AskComposer	Textarea with language hint, example chips, consent line, submit.	onSubmit, disabled	empty, typing, over-limit (600 chars), submitting
PipelineStepper	Shows the four labelled stages while /ask/stream runs.	stage, error	pending, active, done, failed, waking-server
VerdictHero	Icon, headline, one-line summary, evidence-strength meter.	verdict, summary, strength	5 verdicts, loading skeleton, review-flag
VerdictBadge	Compact icon + label chip.	verdict, size	5 verdicts; sm / md
ClaimRow	One atomic claim with status chip and "See proof" action.	claim, status, onOpen	supported, contradicted, partial, unverified
EvidencePassport	Authority, document, version, page, quote with highlight, validated tick, freshness, source link.	evidence	validated, stale, superseded, loading, error
QuoteBlock	Typeset blockquote with , copy button, page jump.	text, highlightRange	collapsed (3 lines) / expanded
FreshnessPill	Current / expiring / stale / superseded with date.	status, date	4 states with icons
NextQuestionChips	Tappable questions that resolve unknown conditions.	questions, onAnswer	idle, answering, answered
PathwayGraph	React Flow graph of routes with status-coloured nodes.	routes, selected	selected, dimmed, keyboard-focus, mobile list mode
TraceTimeline	Per-condition PASS / FAIL / UNKNOWN with quote link.	trace	collapsed by route; expandable

Component	Purpose	Key props	States
AbstainPanel	Respectful "We could not verify this" layout.	missing, authorityHint	with / without suggestions
ReportSheet	Printable family report layout.	report	screen vs print
RuleCandidateCard	Candidate rule with status, maker, validation results.	rule	draft, in_review, approved, rejected
SourcePageViewer	Page text with highlighted quote and zoom.	page, quotes	text mode; image mode (stretch)
ConditionEditor	Edit one leaf condition (attr, op, value, quote).	node, dictionary	valid, invalid, needs-new-attribute
MetricTile / AblationChart / FailureTable	Eval Lab primitives.	metric, series	loading, empty, comparison
Toast, Dialog, Drawer, Tabs, Tooltip, Select, Switch	shadcn/Radix primitives themed with tokens.	standard	keyboard and screen-reader complete

6.7 Microcopy and i18n

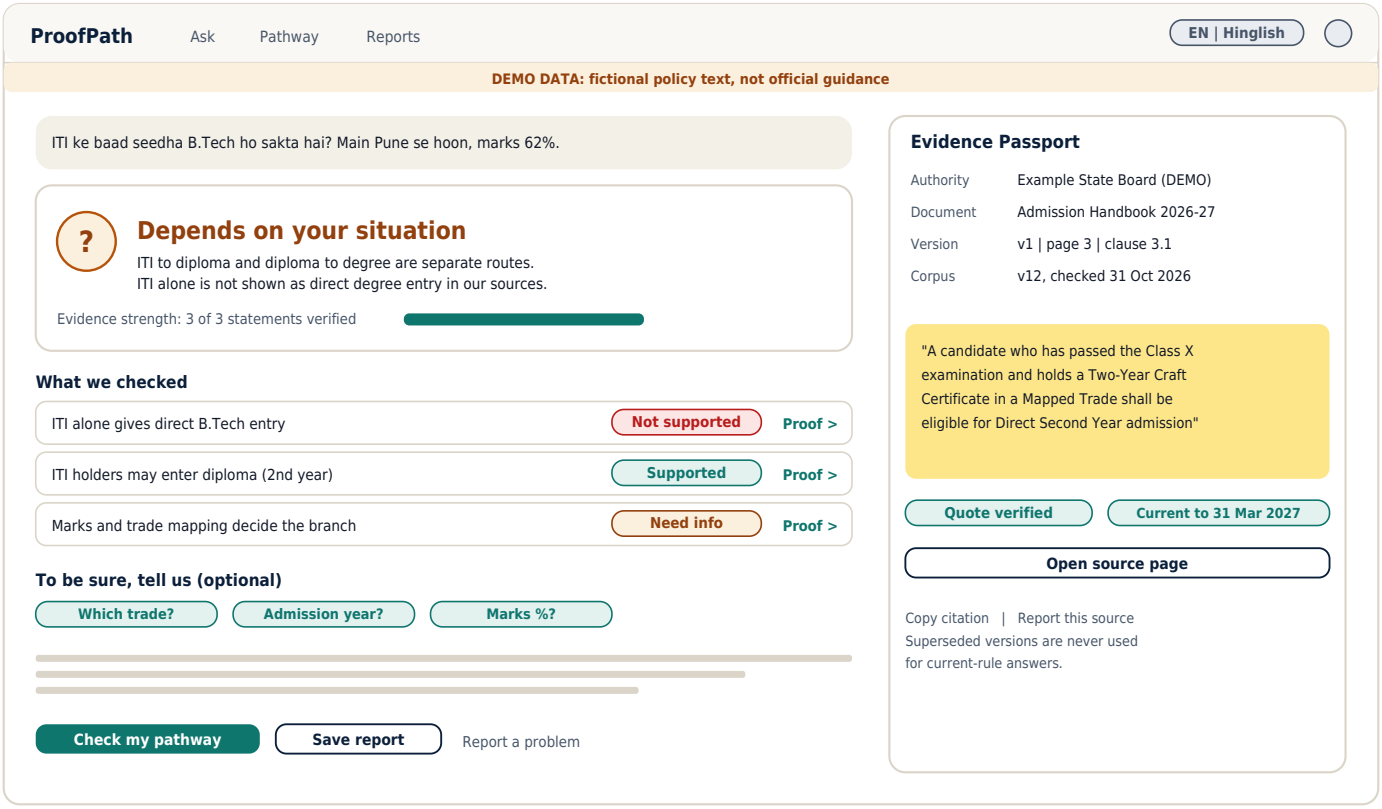
- All UI strings live in locales/*.json under feature namespaces (ask, verdict, pathway, report, studio, errors). No hard-coded strings in components.
- Locales: en and hinglish (Roman script) are written by the team; hi (Devanagari) and mr are supplied by native reviewers and are optional. Missing keys fall back hinglish to en.
- User-visible answer language follows the question language (detected in Understand). UI language is a separate toggle.
- Dates use the locale (en-IN); numbers use Indian grouping; marks use tabular figures.
- Error copy is human: "Server is waking up, this can take a minute" rather than "502".
- Example Hinglish strings: ask.placeholder = "Apna sawaal likhiye, jaise: ITI ke baad diploma kaise milega?"; abstain.title = "Hum isko verify nahi kar paaye"; pathway.disclaimer = "Yeh sirf record par maujood conditions check karta hai. Yeh admission ka faisla nahi hai."

> **Reviewer-supplied Devanagari**
> Do not machine-translate Hindi or Marathi policy terms. Create the en and hinglish files first, mark every key needing a native review in a locales/REVIEW.md checklist, and ship Devanagari only when a reviewer has signed off.

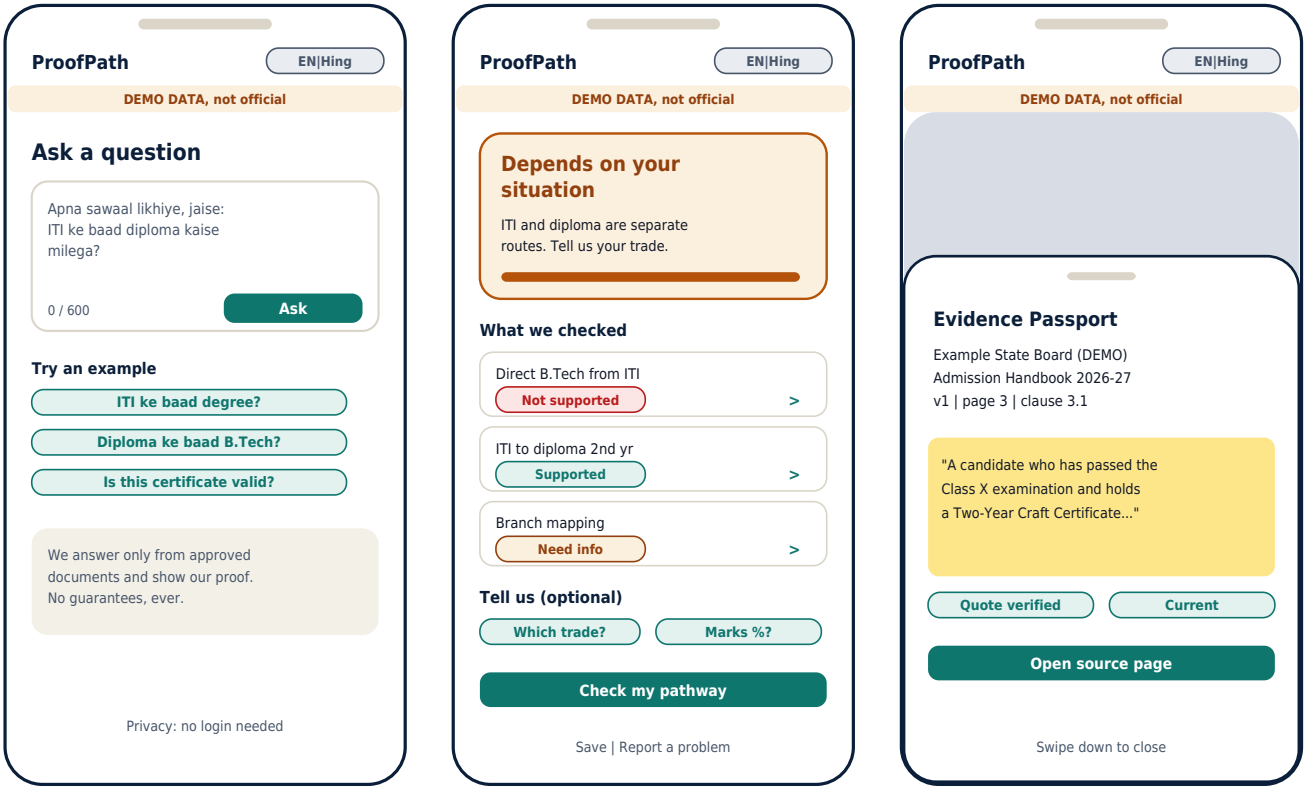
Part 7. Screen-by-screen UI specification

Every screen below lists purpose, layout, components, states, interactions, data and acceptance checks. Each screen must ship with loading, empty, error, abstain (where relevant) and success states, in light and dark mode, at 360 px and 1280 px.

7.1 Answer screen (the hero of the product)



Wireframe, desktop. Left: answer. Right: sticky Evidence Passport for the selected claim. Verdict and summary are above the fold; proof is always one click away.



Wireframe, mobile (360 px): Ask, Answer, and Evidence Passport as a bottom sheet. On mobile the passport opens as a bottom sheet with drag-to-close and focus trap.

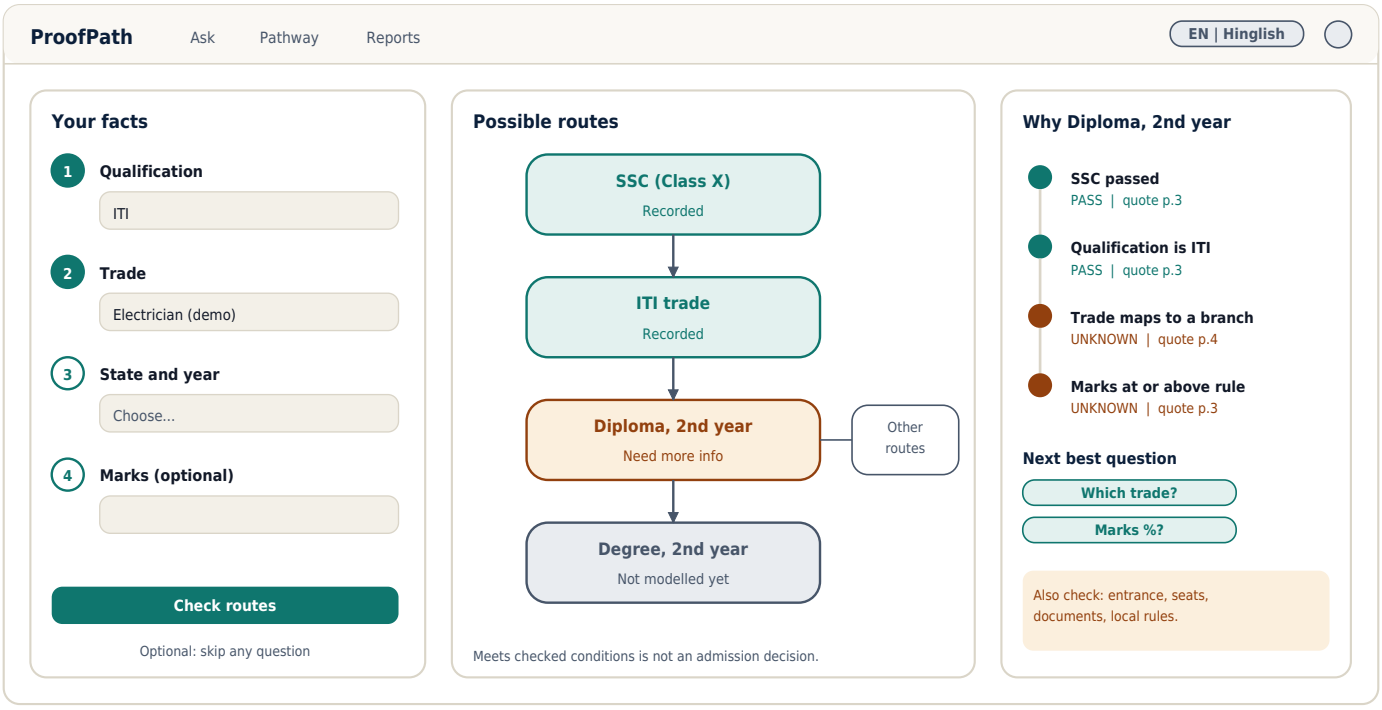
7.2 Landing and Ask

- **Landing**: calm hero in the display serif: "Ask. Verify. Decide." with the sentence "Answers from official documents, with the proof beside every statement." Primary action opens the composer inline. Under it, three steps (Ask, We check sources, You see the proof) and a "What we will never do" card (no guarantees, no ranking by payment, no data selling).
- **AskComposer**: single textarea, 600 character cap with counter, auto-detected language chip, three example chips in Hinglish and English, one-line privacy note ("Do not enter names or phone numbers"). Enter submits; Shift+Enter adds a line. Disabled state while submitting.
- **While running**: PipelineStepper with four labelled stages fed by SSE. If the server is cold, show "Waking up the server, this can take about a minute" with a gentle progress animation; never an empty spinner.
- **States**: empty, typing, over-limit, rate-limited (shows Retry-After countdown), offline (queues nothing, explains), server-waking, failure (retry button keeps the text).
- **Data**: POST /ask/stream; GET /meta/corpus for the banner and footer freshness.

7.3 Answer screen details

- **VerdictHero**: icon, serif headline, one-line summary, "Evidence strength: N of M statements verified" with a segmented meter (not a percentage). If human review is flagged, show a neutral note: "Queued for expert review".
- **ClaimRows**: each atomic claim with status chip (Supported, Not supported, Partly, Need info, Unverified) and "Proof" action that selects the claim and opens the passport (sticky panel on desktop, bottom sheet on mobile). Keyboard: Up/Down moves between claims, Enter opens proof.
- **Explanation**: 3 to 5 short bullets from the explainer. Each bullet ends with small footnote markers [1], [2] that focus the matching passport. Language follows the question.
- **NextQuestionChips**: appear for CONTEXT_DEPENDENT; tapping a chip asks one tiny inline question and re-runs the rule engine only (no new LLM call) via POST /pathway/check.
- **Actions**: Check my pathway, Save report (prompts sign-in), Report a problem (feedback dialog), Copy link to this answer (stores only the answer id).
- **AbstainPanel** (INSUFFICIENT_EVIDENCE): headline "We could not verify this", short explanation of what is missing, "Who to ask" (authority name from the intent map, never a made-up URL), and "Try asking this instead" suggestions. It must feel helpful, not like an error.
- **Stale / superseded**: amber FreshnessPill on the affected passport and a note "This source may be out of date; we flagged it for review".

7.4 Pathway Checker



- **Layout**: three zones on desktop (facts form, graph, trace). On mobile: a stepper form, then a vertical route list; the graph becomes list mode with the same statuses; trace opens in a sheet.

- **Form**: progressive steps (Qualification, Trade, State and year, Marks optional). Options come from GET /facts/dictionary so labels and allowed values are data-driven. Every step is skippable.
- **Graph**: React Flow with custom nodes coloured by status, edge labels showing the rule key. Selecting a node loads its trace. Fully keyboard operable (arrow keys move focus between nodes; Enter selects); provide an equivalent list view for screen readers.
- **Trace**: ordered conditions with PASS / FAIL / UNKNOWN icon, human label, and a link that opens the exact quote. UNKNOWN rows include a "Tell us" action.
- **Caveats card** always visible: "Entrance tests, seat availability, documents and local rules are not checked here."
- **Data**: POST /pathway/check on every answered question (debounced 300 ms). Returns routes, trace, eligible_targets, next_questions, corpus_version.

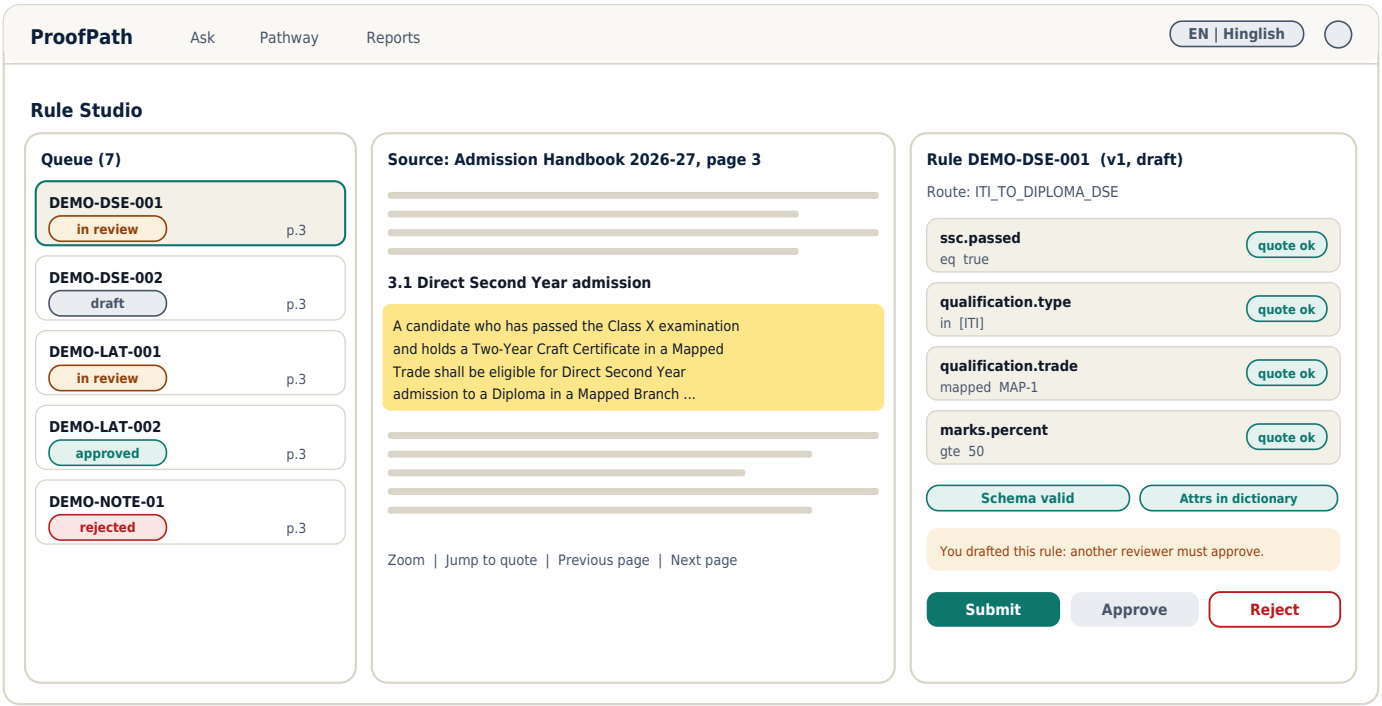
7.5 Family Report

- Generated from an answer and/or pathway result; editable title and notes (local only until saved). Sections: Question, Verdict, What we verified (with sources table), Your routes and status, What is still unknown, Questions to ask the institute, Next steps, Disclaimer and corpus version.
- Screen version is a clean reading layout; print version (print.css) is A4, expands every passport, shows URLs, avoids page breaks inside claim cards. Provide "Print or save as PDF" using the browser print dialog (no server PDF in v1).
- Saved reports list with delete. Reports belong to the owner (object-level authorisation). No sharing links in v1.

7.6 Auth, privacy and settings

- Sign-in and register on one card: email, password, language, guardian-consent checkbox ("I am 18 or older, or a parent or guardian has agreed"). Inline password strength hint; no composition rules beyond length 10+.
- Settings: language, theme, "Allow storing my questions to improve answers" (off by default), export my data, delete my account. Privacy notice in plain language; states that a free-tier LLM provider may use prompts to improve its products and that personal data is scrubbed before sending.
- Session expiry shows a polite re-authentication dialog that preserves the page state.

7.7 Rule Studio (reviewer and admin)



- **Three panes**: queue, source page viewer with the quote highlighted, rule editor. Resizable panes; keyboard shortcuts: J/K next/previous candidate, A approve, R reject (requires reason), E edit, ? shows help.
- **Editor**: route key, title, effective dates, caveats, condition tree as editable rows (all / any groups), each leaf has attribute (from dictionary), operator, value, and its own quote. Live validation chips: schema valid, attributes in dictionary, each quote verbatim.
- **Maker-checker UI**: Approve is disabled for the maker with an inline explanation. Publishing is admin-only and shows a diff against the previous version and the number of cached answers that will be invalidated.

- **Extraction**: "Extract rule candidates from this source version" starts a job; progress and failures are visible; candidates with invalid quotes appear in a "needs fixing" tab rather than being hidden.
- **States**: empty queue, job running, extraction failed, conflict (another reviewer edited), stale source.

7.8 Sources and Source Health

- Sources list: authority, title, version, effective dates, status chip (current, superseded, retired), pages, chunks, last verified, linked rules.
- Upload flow (admin): drag-and-drop PDF, metadata form (authority, title, state, academic year, effective_from/until, URL), checksum shown, size/page limit errors explained.
- Source Health: table of sources expiring within 60 days, superseded versions still referenced by published rules, rules without a current source, and failed ingestion jobs; each row has a primary action.

7.9 Eval Lab



- Run configuration: benchmark set, systems (B0 plain LLM, B1 vanilla RAG, S full), mode (live or replay), ablation toggles.
- Results: MetricTiles, grouped comparison chart, ablation table, per-case table with filters (failed only, intent, category), drilldown showing the trace, evidence and gold labels side by side, and a human-audit form to label unsupported claims.
- Export results as CSV and Markdown for the report. Always show the corpus_version, model name, prompt version and date of each run.

7.10 Admin: users, audit, system

- Users: invite reviewer or admin by email token, change role, disable. Audit log: filterable table with actor, action, object and payload diff; "Verify chain" button runs /admin/audit/verify and shows the first broken row or a green confirmation.
- System page: corpus_mode switch (admin only, with confirmation), corpus_version, LLM budget used today, cache stats, build info.

7.11 System states every page must handle

State	Treatment
Loading	Skeleton matching the layout; stepper for the pipeline.
Empty	One sentence explaining why plus the primary next action.
Error (network, 5xx)	Human message, request id, retry that preserves input.
Rate limited (429)	Countdown from Retry-After; offer to continue later.
Unauthorized / forbidden	401 re-auth dialog; 403 page explains which role is needed.
Not found	Friendly 404 with Ask CTA.

State	Treatment
Offline	Banner; cached last answer remains readable.
Server waking (free host)	Dedicated waking state with realistic time hint.
Stale corpus	Footer shows corpus version and date; amber note if any referenced source is expiring.